

MANUAL TÉCNICO

Sistema de Análisis de Rendimiento Académico

Versión: 1.0

Curso: Lenguajes Formales y de Programación

Sección: -B

Fecha: 02 Marzo 2026

TABLA DE CONTENIDO

- Informacino General
- Introducción
- Estructura del Programa
- Estructuras de Datos (Clases)
- Métodos Principales
- Lógica de Lectura y Procesamiento de Archivos
- Algoritmos de Reportes
- Cálculos Estadísticos
- Requerimientos Técnicos
- Observaciones del Desarrollo
- Diagramas de Flujo
- Pseudocódigo
- Conclusión

1. INFORMACIÓN GENERAL

Proyecto	Sistema de Análisis de Rendimiento Académico
Lenguaje	C++ (Estándar C++11)
IDE	CLion
Compilador	MinGW (Windows) / g++ (Linux/macOS)
Arquitectura	Programación Orientada a Objetos
Repositorio	LFP_-B_1S2026/Practical

INTRODUCCIÓN

El Sistema de Análisis de Rendimiento Académico es una aplicación desarrollada en C++ que permite procesar y analizar datos académicos de estudiantes, cursos y notas. El sistema implementa tres fases fundamentales:

Fase	Descripción
Léxica	Validación de tokens (números, strings, formatos específicos)
Sintáctica	Verificación de estructura (campos separados por comas)
Semántica	Validación de reglas de negocio (rangos, carreras válidas, etc.)

El programa genera 5 reportes estadísticos en formato HTML para facilitar la visualización y toma de decisiones académicas.

ESTRUCTURA DEL PROYECTO

```

└─ Practica1/
|
|
└─ └─ cmake-build-debug/      # Carpeta de compilación (automática)
|
|
└─ └─ data/                  # Archivos .lfp (SE DEBEN CREAR)
|   └─ └─ estudiantes.lfp
|       └─ └─ cursos.lfp
|           └─ └─ notas.lfp
|               └─ └─ carreras.lfp
|                   └─ └─ cursos_validos.lfp
|
|
└─ └─ include/               # Archivos de cabecera (.h)
|   └─ └─ analisisCarrera.h
|       └─ └─ curso.h
|           └─ └─ CursosMasReprobacion.h
|               └─ └─ estudiante.h
```

- | |— generalesCursos.h
- | |— lectorArchivos.h
- | |— mejoresEstudiantes.h
- | |— nota.h
- | |— rendimientoEstudiantes.h
- | |— validaciones.h
- | |— validacionLexica.h
- |
- |—  reportes/ # Implementación de reportes
- | |— analisisCarrera.cpp
- | |— CursosMasReprobacion.cpp
- | |— generalesCursos.cpp
- | |— mejoresEstudiantes.cpp
- | |— rendimientoEstudiantes.cpp
- |
- |—  CMakeLists.txt # Configuración de compilación
- |—  lectorArchivos.cpp # Lectura de archivos
- |—  main.cpp # Compilacion
- |—  validaciones.cpp # Validaciones Semanticas
- |—  validacionLexica.cpp # Validación Lexicas

ESTRUCTURAS DE DATOS (CLASES)

El programa utiliza structs (simulando clases simples) para representar las entidades principales:

Struct Estudiante

```
struct Estudiante {  
    int carnet;  
    string nombre;  
    string apellido;  
    string carrera;  
    int semestre; // 1-10  
};
```

Archivo: include/estudiante.h

Struct Cursos

```
struct Cursos {  
    int codigo; //numero entero  
    string nombre;  
    int creditos; //numero entero de 1-8  
    int semestre; // 1-10  
    string carrera; //carrera que pertenece el curso  
};
```

Archivo: include/curso.h

Struct Notas

```
struct Notas {  
    int carnet;  
    int codigoCurso;  
    double nota; // 0 - 100  
    string ciclo; // 1S - 2S  
    int año; // año en que se cursó número entero  
};
```

Archivo: include/nota.h

Estructuras Auxiliares (para reportes)

```
// Para reporte de cursos con reprobación
struct CursoReprobados {
    Cursos c;           // Curso
    double media;       // Porcentaje de reprobación
    int repro;          // Cantidad de reprobados
    int aproba;         // Cantidad de aprobados
    int cantidad;       // Total estudiantes
};

// Para reporte de mejores estudiantes
struct Promedios {
    Estudiante studens; // Estudiante
    double promedio;     // Promedio general
};
```

MÉTODOS PRINCIPALES

Módulo de Carga de Archivos (lectorArchivos.cpp)

Método	Descripción	Parámetros	Retorno
cargarEstudiante	Carga y valida estudiantes	nombreArchivo, listaEstudiantes	vector<Estudiante>
cargarCursos	Carga y valida cursos	nombreArchivo, listaCursos	vector<Cursos>
cargarNotas	Carga y valida notas	nombreArchivo, listaNotas	vector<Notas>
imprimirPantalla (sobrecargado)	Muestra datos en consola	lista (3 tipos)	void

Módulo de Validación Léxica (validacionLexica.cpp)

Método	Descripción	Validación
eliminarEspacio	Elimina espacios, tabs, saltos de línea	-
esStringLexico	Verifica que solo contenga letras	[a-zA-Z]+
esIntLexico	Verifica que solo contenga dígitos	[0-9]+
esDecimalLexico	Verifica formato decimal	[0-9]+([0-9]+)?
identificadorCurso	Valida nombre de curso	Letras y números en orden
identificadorCiclo	Valida formato de ciclo	[1-2][Ss]
identificadorAnio	Valida formato de año	[0-9]{4}

Módulo de Validación Semántica (validaciones.cpp)

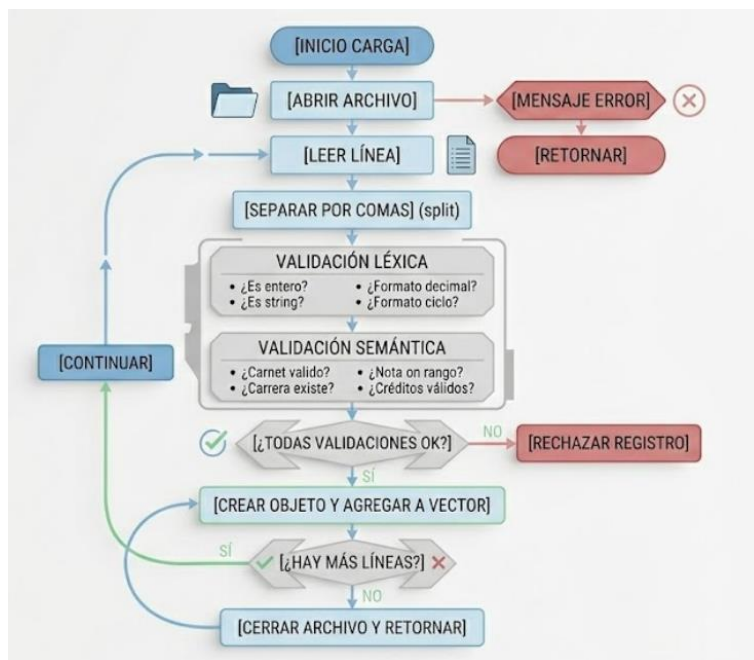
Método	Descripción	Regla
validacionCarnet	Valida longitud de carnet	9 dígitos
nombreValido	Valida longitud nombre	> 3 caracteres
apellidoValido	Valida longitud apellido	> 3 caracteres
carreraValidaUsac	Valida carrera existente	Industrial, Civil, Sistemas, etc.
esSemestreValido	Valida rango semestre	1-10
validacionNombreCurso	Valida numeración del curso	1-5
validacionCodigoCurso	Valida código	3 dígitos
validacionCreditos	Valida créditos	1-8
validacionNota	Valida rango nota	0-100
validacionCiclo	Valida ciclo	1S o 2S

Módulo de Reportes (reportes/*.cpp)

Método	Reporte	Archivo generado
generarReporteCursos	Estadísticas por Curso	estadisticasCursos.html
generarReportesRendimiento	Rendimiento por Estudiante	rendimiento.html
generarReporteMejoresEstudiantes	Top 10 Mejores	mejoresEstudiantes.html
generarReportesMasReprobados	Cursos con Mayor Reprobación	CursosMasReprobados.html
generarReportesAnalisisCarrera	Análisis por Carrera	ReportesAnalisisCarrera.html

LÓGICA DE LECTURA Y PROCESAMIENTO DE ARCHIVOS

Proceso General de Carga



Carga de Estudiantes (cargarEstudiante)

Pseudocódigo:

Abrir archivo estudiantes.lfp

SI archivo no se abre: mostrar error y retornar lista vacía

Mientras haya líneas:

- a. Leer carnet (hasta coma)
- b. Leer nombre (hasta coma)
- c. Leer apellido (hasta coma)
- d. Leer carrera (hasta coma)
- e. Leer semestre (hasta fin de línea)

- f. Limpiar espacios del semestre

// VALIDACIÓN LÉXICA

Si carnet no es entero: error

Si nombre no es string: error

Si apellido no es string: error

Si carrera no es string: error

Si semestre no es entero: error

Si alguna validación léxica falla:

 marcar registro como inválido

Si pasa la validación léxica:

// VALIDACIÓN SEMÁNTICA

SI carnet válido Y nombre válido Y apellido válido

Y carrera válida Y semestre válido:

 Crear objeto Estudiante

 Asignar valores (convertir a int)

 Agregar a lista

 Mostrar "Agregado exitosamente"

SINO:

 Mostrar "Registro rechazado"

Reiniciar bandera de error

Cerrar archivo

Retornar lista

Carga de Cursos (cargarCursos)

Pseudocódigo:

Abrir archivo cursos.lfp

Si archivo no se abre: mostrar error y retornar lista vacía

MIENTRAS haya líneas:

- Leer código (hasta coma)
- Leer nombre (hasta coma)
- Leer créditos (hasta coma)
- Leer semestre (hasta coma)
- Leer carrera (hasta fin de línea)
- Limpiar espacios de carrera

// VALIDACIÓN LÉXICA

Si código no es entero: error

Si nombre no es identificadorCurso: error

Si créditos no es entero: error

Si semestre no es entero: error

Si carrera no es string: error

Si alguna validación léxica falla:

 marcar registro como inválido

Si pasó validación léxica:

 // VALIDACIÓN SEMÁNTICA

 SI código válido Y nombre válido Y créditos válidos

Y semestre válido Y carrera válida:

Crear objeto Cursos

Asignar valores (convertir a int)

Agregar a lista

SINO:

Rechazar registro

Reiniciar bandera

Cerrar archivo

Retornar lista

Carga de Notas (cargarNotas)

Pseudocódigo:

Abrir archivo notas.lfp

Si archivo no se abre: mostrar error y retornar lista vacía

Mientras haya líneas:

- Leer carnet (hasta coma)
- Leer código curso (hasta coma)
- Leer nota (hasta coma)
- Leer ciclo (hasta coma)
- Leer año (hasta fin de línea)
- Limpiar espacios del año

VALIDACIÓN LÉXICA

Si carnet no es entero: error

Si código no es entero: error

Si nota no es decimal: error

Si ciclo no es identificadorCiclo: error

Si año no es identificadorAnio: error

Si alguna validación léxica falla:

 marcar registro como inválido

Si pasó validación léxica:

 // VALIDACIÓN SEMÁNTICA

 SI carnet válido Y código válido Y nota válida

 Y ciclo válido:

 Crear objeto Notas

 Asignar valores (convertir a int/double)

 Agregar a lista

 SINO:

 Rechazar registro

Reiniciar bandera

Cerrar archivo

Retornar lista

ALGORITMOS DE REPORTE

reporte 1: estadísticas por curso

por cada curso agarramos todas sus notas, calculamos el promedio, la nota más alta, la más baja, la mediana (el valor del centro) y la desviación estándar (qué tan dispersas están las notas). todo se guarda en un html.

reporte 2: rendimiento por estudiante

por cada estudiante juntamos sus notas, vemos cuántas pasó (61 o más) y cuántas quedó, sumamos los créditos de los cursos que aprobó y sacamos su promedio. se guarda en un html.

reporte 3: top 10 mejores estudiantes

calculamos el promedio de cada estudiante, los ordenamos de mejor a peor y mostramos los primeros 10. los primeros 3 llevan medallita 🏆 🥈 🥉 . todo al html.

reporte 4: cursos con mayor reprobación

por cada curso vemos cuántos estudiantes lo perdieron, calculamos el porcentaje de reprobados y ordenamos de más reprobados a menos. solo mostramos los que tienen al menos un reprobado. al html.

reporte 5: análisis por carrera

por cada carrera (sistemas, civil, etc.) juntamos sus estudiantes, calculamos el promedio general de la carrera, contamos cuántos estudiantes hay por semestre y vemos cuántos cursos tiene esa carrera. todo bonito en html.

Requerimientos de Software

Componente	Versión	Observación
Lenguaje	C++11	Estándar utilizado
Compilador	g++ 5.0+ / MinGW	C++11 compatible
IDE	CLion 2020.1+	Recomendado
Sistema Operativo	Windows/Linux/macOS	Multiplataforma

Requerimientos de Hardware

Componente	Mínimo	Recomendado
Procesador	1.5 GHz	2.5 GHz+
RAM	2 GB	4 GB+
Disco	100 MB	500 MB

Librerías Utilizadas

Librería	Propósito
<iostream>	Entrada/salida por consola
<fstream>	Lectura/escritura de archivos
<string>	Manejo de cadenas
<vector>	Almacenamiento dinámico

<code><algorithm></code>	Ordenamiento (sort)
<code><cmath></code>	Operaciones matemáticas (sqrt, pow)
<code><map></code>	Estructura clave-valor (para semestres)
<code><cctype></code>	Validación de caracteres

OBSERVACIONES DEL DESARROLLO

Decisiones de Diseño

Uso de struct en lugar de class: Simplicidad, no se requiere encapsulamiento complejo.

Validación en tres fases:

- Léxica: Verifica que los tokens tengan el formato correcto
- Sintáctica: Implícita en el orden de lectura (campos separados por comas)
- Semántica: Verifica reglas de negocio (carreras válidas, rangos, etc.)
- Manejo de errores: Se implementó validación robusta con mensajes descriptivos.
- Rutas relativas: ../data/ para archivos de entrada, facilitando portabilidad.

Generación HTML: Formato simple pero funcional, sin dependencias externas.

Limitaciones Conocidas

Limitación	Descripción	Posible mejora
Sin manejo de excepciones	No se usa try-catch	Agregar manejo de excepciones
Ordenamiento burbuja	Ineficiente para muchos datos	Implementar sort de STL
Rutas fijas	No permite rutas personalizadas	Pedir rutas al usuario
Sin validación de existencia	No verifica que cursos/estudiantes existan antes de procesar notas	Agregar validación cruzada